Portfolio Optimisation *with* Derivatives & Structured Products

BEYOND MEAN–VARIANCE

Mental Accounting Framework

From **my MSc Finance thesis** on mental-account portfolio optimisation with derivatives and structured products — and its original **R-based optimisation tool** — to a **scalable Monte-Carlo + CVaR engine**. An interactive research optimiser that builds goal-based portfolios across **four precision modes** and **four risk-based approaches**, prices derivatives and structured products, back-tests out-of-sample, and now scales to large portfolios of **many assets and many derivatives** at once.

4 risk-based methods
VaR · Expected-Shortfall · rigorous-ES · Monte-Carlo CVaR — with configurable parameters that set the risk appetite

16 instruments
options, spreads, collars, capital-guaranteed & barrier notes

≈ 60× faster
Turbo solver — one of four grid resolutions

Multi-derivative
several derivatives optimised jointly · scales O(S·N)

10,000+ tickers
live market data — equities, ETFs & crypto

AI-powered
natural-language explanations & Q&A on every input

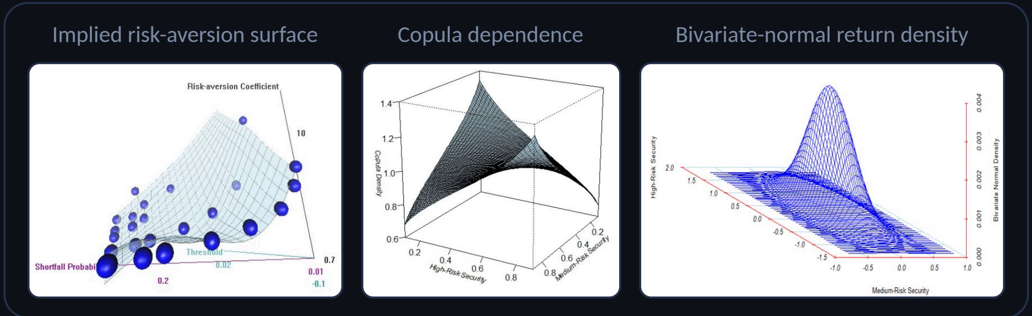
Callable engines · REST API + MCP
optimisation & back-testing engines callable by external portfolio, risk & trading systems — or an AI agent

01 Grounded: two seminal papers

Built on two seminal papers, both replicated in my **2012 MSc thesis** (USI Lugano):

- **Das, Markowitz, Scheid & Statman (2010)** — the **mental-accounts framework**: sub-portfolios as goals, each with a threshold *H* and shortfall probability *α*; the mean-variance ↔ mental-accounting **equivalence** and an implied risk aversion per goal
- **Das & Statman (2009)** — the optimisation algorithm for **derivatives & non-normal returns** in mental accounts, tested on **9 structured instruments**
- My thesis **replicated both methodologies** and extended the research to further optimisation scenarios — varying instrument parameters to confirm the equivalence and implied risk aversions, and to show the **potential benefits of adding complex instruments with non-normal return distributions**

SELECTED FIGURES FROM MY THESIS

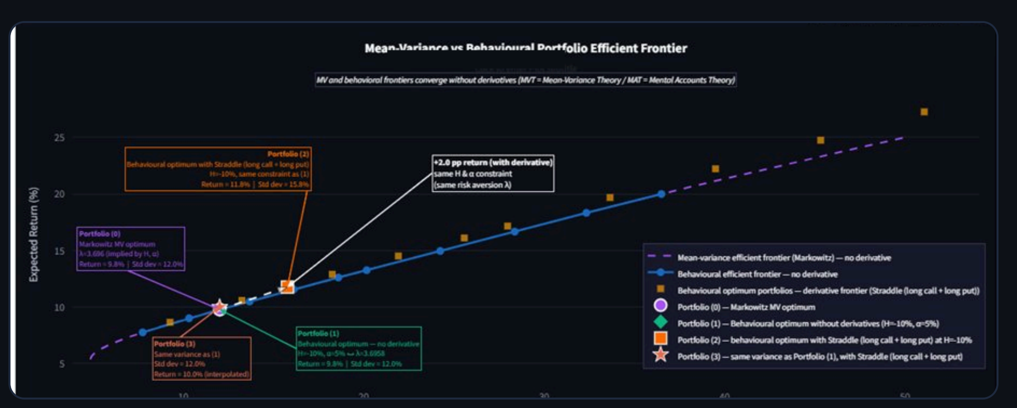


02 Rebuilt: a fully automated optimiser

My R thesis tool is now re-built and fully automated in **Python**: an **exact grid optimiser** that runs every simulation — on the thesis inputs or **real market data** — reproduces all the thesis results, and adds **AI-powered explanations** throughout.

- **Three risk approaches** — VaR, thesis-faithful ES, and a rigorous ES mode
  each with a configurable loss threshold and probability of loss
- **Four precision modes** — Fast / Standard / High + a custom **Turbo** solver (~60×)
  from a 21×15 grid (Fast) up to 51×99 (High) — over 20 billion state combinations at full resolution
- Enhances the optimisation by up to **+2.4 pp** of return for the same risk level

OPTIMISER OUTPUT · SAMPLE RUN



03 Extended: Derivatives & structured products

Extended the library to **sixteen instruments** with adjustable parameters, priced consistently via Black–Scholes — vanilla options, **collars, capital-guaranteed & barrier notes, spreads, certificates** — plus a **builder to configure your own**.

Priced arbitrage-consistently — and you can build and configure new instruments. The same engine drives the optimiser, the back-test and the Monte-Carlo tab.

04 Added: Out-of-sample back-test

Does the optimisation hold up out-of-sample? Build optimal weights on a construction window, then **buy-and-hold** across an evaluation window — marking the derivative **to market** each step — to **test the efficiency of the optimisation methods** and read off the realised **alpha and beta** of each holding and of the portfolio.

Same portfolio held out-of-sample, with vs without a protective put — here the put cushions the drawdowns and finishes ahead. You can back-test any instrument over any window.

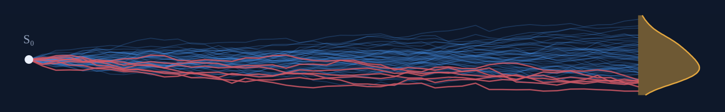
05 Scaled: a separate Monte-Carlo + CVaR optimiser for institutional-scale portfolios

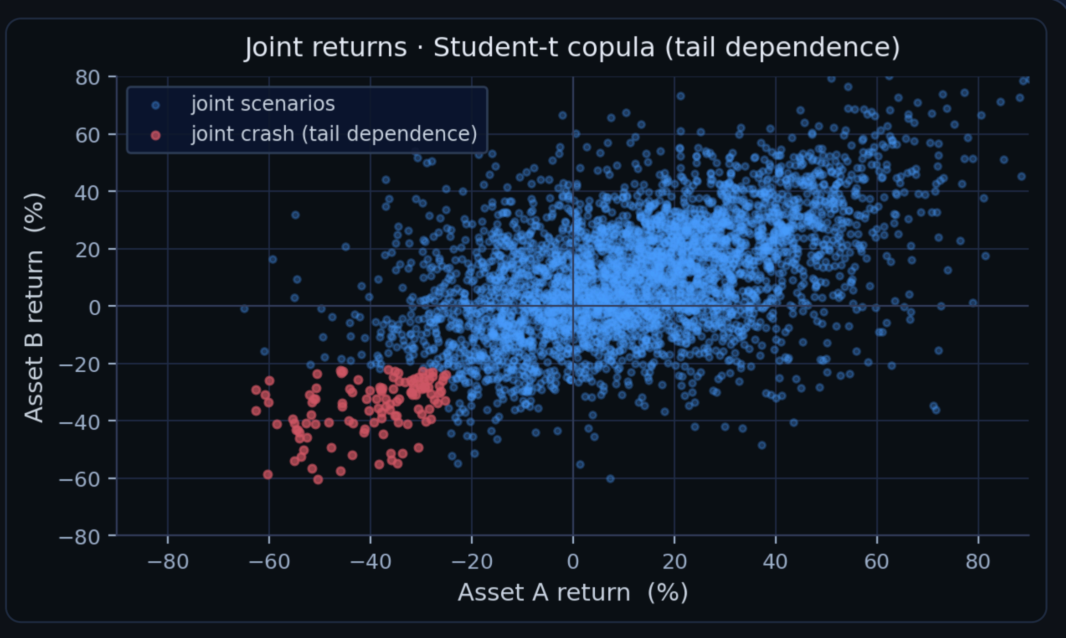
The grid is exact but its state space grows as m^N — impossible past ~4 assets. The new engine instead **samples S joint scenarios** through a copula and solves the goal as a **linear program**.

- Cost is **linear in the number of assets** — and free in the number of scenarios
- **Several derivatives at once**, even on different underlyings
- Gaussian or **Student-t copula** for tail dependence (assets crash together)
- Coherent **Expected-Shortfall** floor via Rockafellar–Uryasev

$$\max\ E[r]\ \text{ s.t. }\ \zeta + 1/(\alpha S)\cdot\Sigma\, z_s \le -L,\ \ z_s \ge -w\cdot R_s - \zeta,\ \ \Sigma w = 1$$

The result: a **goals-based optimiser at institutional scale** — dozens of holdings and several overlays priced consistently, solved in a single linear program.





Scenario engine — joint return draws from a Student-t copula (two assets shown). The red lower-left cluster is the joint-crash tail dependence a Gaussian copula misses; thousands of such scenarios feed the α-CVaR linear program.

WHY IT MATTERS · EXACT GRID VS SCALABLE ENGINE

Exact grid
- Bit-exact, **thesis-grade** reproduction
- Practical only to **≈4 assets** (state space ~ m^N)
- One derivative at a time

Scalable engine
- **Institutional scale** — cost linear in N
- **Dozens of holdings**, several overlays at once
- α-CVaR via one linear program

OPTIMAL WEIGHTS · SCALABLE CVAR OPTIMUM

AAPL	72.8%
MSFT	22.7%
Protective put · AAPL	4.5%

E[r] 24.6% · realised ES −20.0% (α 5%) · vol 25.5% · skew +0.30. JPM / TLT / XLE / GLD priced 0 at this floor — the put on AAPL lets the optimiser hold the high-return tech sleeve within the tail-risk budget.

06 Validated: against theory and out-of-sample

Every figure is checked, not asserted — validated against the original thesis and against closed-form theory before any result is shown. The scalable engine **complements** the exact reference; it never replaces it.

VALIDATION CHECK	RESULT	
MC optimum vs exact-grid MV frontier	on the frontier to **~2–3 bp**	✓
Scenario Expected-Shortfall vs closed-form Normal	**−28.4%** vs **−28.6%**	✓
Frontier behaviour	monotone · ES floor **binds exactly** · infeasible floors flagged	✓
Thesis result tables (R → Python port)	reproduced **within discretisation noise**, end-to-end	✓
Out-of-sample back-test	weights held through a later window · derivative marked to market	✓

07 Engineered: a clean pipeline, AI-powered

A clean numerical pipeline — **both engines** share the same stages, differing only in the optimiser step:

Universe + market data → Black-Scholes pricing → Optimiser grid VaR/ES · MC CVaR-LP → Efficient frontier → Interactive app · PDF

Exact grid: VaR / Expected-Shortfall weight search. Scalable: copula scenarios solved by a CVaR linear program (HiGHS).

- **Live, interactive UI** — real-time charts, weight bars and a one-click **PDF report**
- **AI-powered glossary** explains derivatives, risk measures and portfolio theory in plain language
- **AI-assisted explanations** turn each optimisation result into a readable narrative
- Black–Scholes pricing under every scenario keeps derivative payoffs **arbitrage-consistent**

BUILT WITH

Python · NumPy · SciPy · HiGHS LP · Streamlit · Plotly · Altair · Copulas · Black–Scholes · Rockafellar–Uryasev CVaR · AI · LLM glossary

SJ
Sami Jeddou
Senior Financial Services Transformation — Risk & Capital Markets

Interactive Python · Streamlit app — **user guide available in the app**

Happy to connect and discuss — feel free to reach out